Islamic University of Technology

Lab 12: Pointer Basics

CSE 4108 : Structured Programming Lab

Section 1B

MIA, NFS, IAC, ASH

February 25, 2025

1 Lab Tasks

1.1 Mine Swapper

You have done swapping between two variables before. But writing those 3 lines for every swap required becomes quite cumbersome. What if we had a function that would swap the values stored in two variables? Since C passes arguments by value, we cannot just use normal variables as arguments. Rather, we need to use pointer to those variables.

Write a function in C that would swap the values of two integer variables with the help of a function. You can do the swapping using either temporary variable or by using some arithmetic or logical operation. And again, remember, a function does what it says. So the function itself will only do the swapping. If you have to print the values, do it in the main function. This rule is true for all other problems. The declaration of the function looks like below.

```
1 void swapInts(int *a, int *b);
```

Design the input and output accordingly to match the task.

1.2 A Lot of Printing

Write a C program that would print the **name**, **data type**, **size**, **address of variable**, **value via variable**, **name of a pointer to that variable**, **value of the pointer (address) to that variable** and **value via pointer to that variable** for `char`, `int`, `long long`, `float` and `double variables`.

Define the variables your self and assign some value. You can hard code the values in code, no need to scan. For each variable, print the name, data type and name of the pointer manually. Use `sizeof` operator for size. The output formatting does not need to match exactly.

[Sample Program Output]

Name	Type	Size	VarAdd	VarValue	PtrName	PtrVal	PtrDeref
c	char	1	000000241E2FF973	a	p_c	000000241E2FF973	a
i	int	4	000000241E2FF964	5	p_i	000000241E2FF964	5
ll	long long	8	000000241E2FF950	100	p_ll	000000241E2FF950	100
flt	float	4	000000241E2FF944	2.4000	p_flt	000000241E2FF944	2.4000
dbl	double	8	000000241E2FF930	5.2000	p_dbl	000000241E2FF930	5.2000

[Hint] *Use* `%zu` *conversion specification for printing size of variables directly. Use* `%lld` *for printing* `long long` *and use* `%p` *for printing addresses/pointers.*

1.3 Something Related to GCD Again

In an earlier lab, you already learnt how you can reduce a fraction given the numerator and denominator of the fraction separately. Now it is time to convert it to a function.

Write a function in C that would take the numerator and denominator of a function as input and then give you the reduced numerator and denominator. Since you require two things back from the function, you need to use pointers. The declaration of the function is like below.

```
1 void reduceFraction(int nume, int denom, int* ptr_rnume, int* ptr_rdenom);
```

Design the input and outputs accordingly. Remember to keep the printing in main function.

[Hint] *If you have forgotten, you can use GCD to reduce fractions. And you can make a separate function to calculate GCD, if you want to keep everything clean and professional.*

1.4 Back to Texas

Convenience Delivery Shop (CDS) of Imperial University of Texas (IUT) is a small retail business that stocks a range of everyday items such as snack foods, confectionery, soft drinks, toiletries, etc. Everyday numerous students get their daily goods from CDS. After purchasing an item, whenever a student gives a note of large denomination (like $500 for example), the shop owner Hashin bhai faces a huge trouble. He can easily calculate the amount to be returned. But he does not know how to pay that amount using smallest number of denominations.

Hashin bhai knows you're a great programmer and asked for your help. You need to write a function that takes U.S. dollar amount as input and then gives back the smallest number of $20, $10, $5, and $1 bills required to pay the amount. Hashin bhai has ensured you that he will not enter any fractional value as input.

Again, the function gives back the amount of different bills. Since there are 4 different types of bill amounts to give back, pointers are required. The declaration of the function is like below.

```
1 void getBillAmounts(int dollars, int *twenties, int *tens, int *fives, int *ones);
```

Do the scanning and printing in main function.

[Sample Program]
Input

```
Enter a dollar amount: 93
```

Output

```
$20 bills: 4
$10 bills: 1
$5 bills: 0
$1 bills: 3
```

[Hint] *Divide the amount by 20 to determine the number of $20 bills needed, and then reduce the amount by the total value of the $20 bills or use remainder operator. Repeat for the other bill sizes in descending order. Be sure to use integer values throughout, not floating-point numbers.*

1.5 Mid Term

Write a function in C that would take an array which is already sorted in ascending order, the array length and a pointer as input and return a pointer to the median of the value(s).

Already sorted in ascending order means you do not need to worry about sorting the array yourself. The values will be sorted from small to large when sent to the function.

Now even length arrays basically have two medians, the middle two elements. Generally their average is taken. But since we are returning pointer to the median, we need to inform one more thing. Whether there is a second median or not, in other words, whether the array is of even length or not. If there is a second median, we can add 1 with the returned pointer (which points to the first median) to get the second median pointer. So, the function will additionally take an integer (Boolean) pointer as input and return 1 (true) if there is a second median or 0 (false) if there is not. The declaration of the function looks like below.

```
1 double* getPtrToMedian(double arr[], int len, int *secondMedian);
```

To clarify, if the function is called on, {`1 3 4 5 6 7`}, it will return pointer to the 3rd element with value 4 and set `*secondMedian` to 1. But if the function is called on {`20 40 50`}, it will return pointer to the 2nd element 40 and set `*secondMedian` to 0.

Design the input and output accordingly to test your program. Again, do the scanning and printing in main function.